JavaScript

Definition of JS

JavaScript is a scripting or a programming language which is used to add some basic functionality to web pages.

History of JS

- Invented by Brenden Eich in the year of 1995.
- There was a company called Netscape Corporation, they had their own browser called Netscape Navigator.
- But the problem with this browser is that only supported static pages (web page developed using html and css or which does not have any functionality in it).
- Now the company wanted to add some scripting language to browser to support dynamic web pages (web pages which has some functionality or behaviour in it).
- For that they thought of two ways:
    - To collaborate with Sun Microsystem company who introduced Java Programming Language (1995) in order to embed Java into browser.
    - To hire Brenden Eich to embed scheme language (1970).
- Finally, they hired Brenden Eich and he introduced javascript within 9-14 das and gave a first name as “MOCHA” in the year 1995.
- “MOCHA name was not popular at that time so he changed the name to “LIVESCRIPT” in the year 1995
- Even “LIVESCRIPT” was not popular so again he renamed it as “JAVASCRIPT”.
- After some period of time Netscape corporation sold javascript to ECMA company.
- ECMA stands for European Computer Manufacturing Association, they added some standard functionality to JavaScript and then renamed it as ECMA Script and the first version was released in the year of 1997 called as ES-1.
- The standard version of ECMA Script is ES-6 (2015) and the current version is ES 14 (2024, June).

What is JAVASCRIPT?

JS is a scripting or programming language which is used for both frontend & backend

- Scripting & Programming language are different because scripting language does not have a complier and programming language has a complier
- All scripting language can be a programming language but all programming language cannot be a scripting language because scripting language does not have complier.
- Usually script written in JS instruct the browser what kind of action it must do and how to do it.

Characteristics of JS:

- JS is a high-level language or a user-friendly language (programmer can easily read & write the code).
- It is a interpreted language (checks the code line by line and executes).
- It is single threaded language (Only one call stack and one heap memory).
- Loosely typed language (No particular variable to store values).
- It is dynamic language (During runtime we can change the datatype).
- It is synchronous language. (Until the error is resolved the rest of the code won't get executed the command will be stuck at error)
- It is object-based language

Ways to add JS:

- Internal
  - We write the JavaScript code inside the HTML file itself.
  - The code is placed within <script> </script> tags.
  - Usually written inside <head> or at the end of <body>
- External
  - JavaScript code is written in a separate .js file.
  - That file is linked to the HTML using <script src="filename.js"></script>.

Output Methods in JS:

1. console.log() : it is used to print the statement in console.
2. console.error():it is used to print the error message in console. [red color]
3. console.warn():it is used to print the warning message in console. [yellow color]
4. document.write(): it is used to print statement on UI without space between two words.
5. document.writeln(): it is used to print statement on UI with space between two words.
6. alert (): it is used to display a pop-up message on the UI.
7. confirm (): it is used to take confirm message from the user.
8. prompt (): it is used to take the user input.

Tokens:

- Tokens are the smallest unit in the program.
- In tokens we have
  - Keywords
  - Identifiers
  - Literals/Values
  - Operators
  - Statements

1. Keywords:
   a. Keywords are the pre-defined words which has its own meaning.
   b. In JS we have 60+ keywords
   c. Example: [var, let, const, if, else, switch] etc.

2. Identifiers:
   a. Identifiers are the name given to a variable.
   b. Rules of Identifiers:
      i. Identifier should not be a keyword.
      ii. Identifier should not be a number but it can be a alphanumeric.
      iii. There should be no space while defining a identifier either use _ or lowerCameleCase.
      iv. No special character except $ and _.

3. Literals / Values:
    a. Literals are the values assigned to a variable or identifier.

4. Operators:
    a. Operators are the symbols used to perform some operations on the operands.
    b. In Operators we have:
        i. Arithmetic Operator [+, -, *, /, %, **]
        ii. Assignment Operator [=, +=, -=, /=, */]
        iii. Comparison Operator [= =, = = =, >, =, <=,?]
        iv. Logical Operator [&&, ||,]

5. Statements:
    a. A statement are the instructions that are executed by the browser or Node.js.
    b. We have 2 statements in java script.
        i. Conditional Statements:
            1. if: this statement executes a block of code if a specified condition is true.
            2. if else: this statement executes a block of code if a specified condition is true. If the condition is false, another block of code will get executed.
            3. switch case: The switch expression is evaluated once. The value of the expression is compared with the values of each case. If there is a match, the associated block of code is executed. If there is no match, the default code block will get executed.
        ii. Looping Statements:
            1. for: it repeats a loop until a specified condition evaluates too false.

Difference between var, let and const

	var	let	const
Only declaration	possible	possible	not possible
Initialization	possible	possible	not possible
Declaration and initialization	possible	possible	possible
Re-declration	possible	not possible	not possible
Re-initialization and	possible	possible	not possible
Re-declaration and Re-initialization and	possible	not possible	not possible

Hoisting:

- Accessing the variable without declaring and initialization is called hoisting.
- Hoisting is java script default behaviour of moving all declaration to the top of current scope.
- Hoisting will happen only for var keyword.
- If we try to access for let and const it will go to TDZ [Temporal Dead Zone].
- TDZ means that an area where variable is not accessible until you utilization.

Data Types:

Datatypes

Primitive datatypes

i. Number
ii. String
iii. Boolean
iv. BigInt
v. Null
vi. Undefined
vii. Symbol

non-primitive datatypes

i. Functions
ii. Arrays
iii.Objects -→a) math
b) date
c) time

Number Datatype:

- it is a primitive data type in javascript.
- it is used to represent the decimal or integer values.
- it can be positive or negative or zero.

String Datatype:

- it is a primitive data type in javascript.
- it is used to represent the sequence of characters.
- it can be written in single, double quote or backticks.
- If backticks is used we can achieve interpolation and line breaks in printing statements.

Boolean Datatype:

- it is a primitive data type in javascript.
- it is used to represent the true or false values
- it can be true or false

Undefined Datatype:

- it is a primitive data type in javascript.
- it is a default value in javascript given by JS Engine for var declaration.
- when variable is declared but not initialized is called as undefined.
- if variable is printed even when not declared then the output is not defined [Reference Error].

Undefined	Not defined
It's a primitive data type	It's an error.
When we try to print variable declaration, we get undefined	When we try to utilize which is not declared

Null Datatype:

- it is a primitive data type in javascript.
- it is used to represent the absence of any object value.

BigInt Datatype:

- it is a primitive data type in javascript.
- it is used to represent the large integer values.
- The range id from -2^{53} to $+2^{53.}$
- Any number with the suffix of "n" we call it as BigInt.

Typeof Operator:

- typeof is JS keyword that will return the type of a variable when you call it.
- It is special operator in JS.

Variables	Typeof
Number	Number
String	String
Boolean	Boolean
Array	Object
Function	Function
Object	Object
Undefined	Undefined
Null	Object [bug in javascript]

NOTE:

If we use typeof more than once it will get converted internally to string.

Array:

- it is a non-primitive data type in javascript.
- it is a collection of elements of same or different data types.
- it is denoted by square brackets [].
- Refer array inbuilt Methods [Array]

Objects:

- it is a non-primitive data type in javascript.
- it is a collection of key value pairs.
- it is denoted by curly brackets {}.
- Refer object inbuilt methods { Object}

Functions:

- it is a non-primitive data type in javascript.
- Functions are the block of statements which will get executed whenever it called or invoked.

Scopes in Javascript:

i. **Global Scope:** Global variables are those declared outside of a block and can be accessed inside and outside the functions**.**
ii. **Local Scope:** Local variables are those declared inside of a block and it cannot be accessed outside the function.
iii. **Block Scope:** The block scope of a variable means that the variable is accessible within the block that is between the curly braces.
    a. Var- can be accessed inside and outside the block.
    b. Let and const – cannot be accessed outside the block.

Types of Functions [or] Ways to declare a functions in Javascript

1. **Anonymous Function:** a function without a name.
2. **Named Function**: a function with a name or an identifier.
3. **Function with Expression**: Storing a function inside a variable is called as function with expression
4. **First Class Function [or] First Order Function**: Storing a function inside a variable and LHS of the function is called as first class function.
5. **Nested Function**: a function inside another function is called as nested function.
    a. **Closure**: when a child function is accessing the variable which is declared in parent function closure will get formed.
    b. **Javascript Currying**: calling a child function with the parent function with multiple parentheses and also child function should return to parent function.
6. **Arrow Function**: It is used to reduce the regular function. It is introduced in ES6 version [2015]
    a. If we have single parameter parentheses is optional.
    b. If we have a single statement inside the statement body curly braces {} are optional and its support implicit return value.
    c. If we define the curly braces {} implicitly value will not be return, use explicit keyword return.

7. **Immediate Invoke Function with Expression [IIEF]:** calling a function immediate once after the declaration with parentheses. IIFE function will execute only once in web page.
8. **Higher Order Function**: A function that accepts a function as an argument and/or returns a function as its value.
9. **Callback Functions**: A function that's passed as a argument to another function.
10. **Generator Function**: It is used to pause the execution in the mid-way and can return the value and again it will resume from the point where it was be passed.

JSON [JAVASCRIPT OBJECT NOTATION]

- It is format to store and transfer the data.
- JSON is similar to JS object [regular object]
- In JSON both key and value should be in double quotes.
- JSON Provides 2 methods
    - stringify() -> used to convert regular object to JSON object
    - parse() -> used to convert JSON object to regular object.

Synchronous and Asynchronous

Synchronous [Blocking of code]: In synchronous, tasks are executed one after the another in a sequence order, each task must be completed before the next one starts.

Asynchronous [non-blocking of code]: In Asynchronous, task can start and run independently. The program doesn't wait for the task to complete before moving to the next one

setTimeout and setInterval

setTimeout(): this method in JavaScript is used to execute a function or evaluate a string of code after a specified delay.

setInterval(): this method will execute a specified function at set time intervals.

Promises

- Promises is an object that keep track about whether a certain event has happened or not and determines what happened after the event.
- Promises are used to handle the asynchronous functions in javascript.

Promises Instance Methods

- **then ():** is used to handle the resolve state.
- **catch ():** is used to handle the reject state.
- **finally ():** an event whether it is resolve or reject finally will get execute.

Promises Static Methods

- **all ():** it returns array of promises if all the promises are resolved else it returns the first promises which is rejected.
- **any ():** it returns the first promises which is resolved. If all the promises are reject it returns Aggregate Error.
- **allSettled ():**it returns array of promises that are settled.
- **race ():** it returns the first promise which is settled

Fetch Method [fetch()]

- It is used to get the data from the backend.
- Status code 200: means that the request was successful.
- Status code 404: means “Page not found”.

Async and Await:

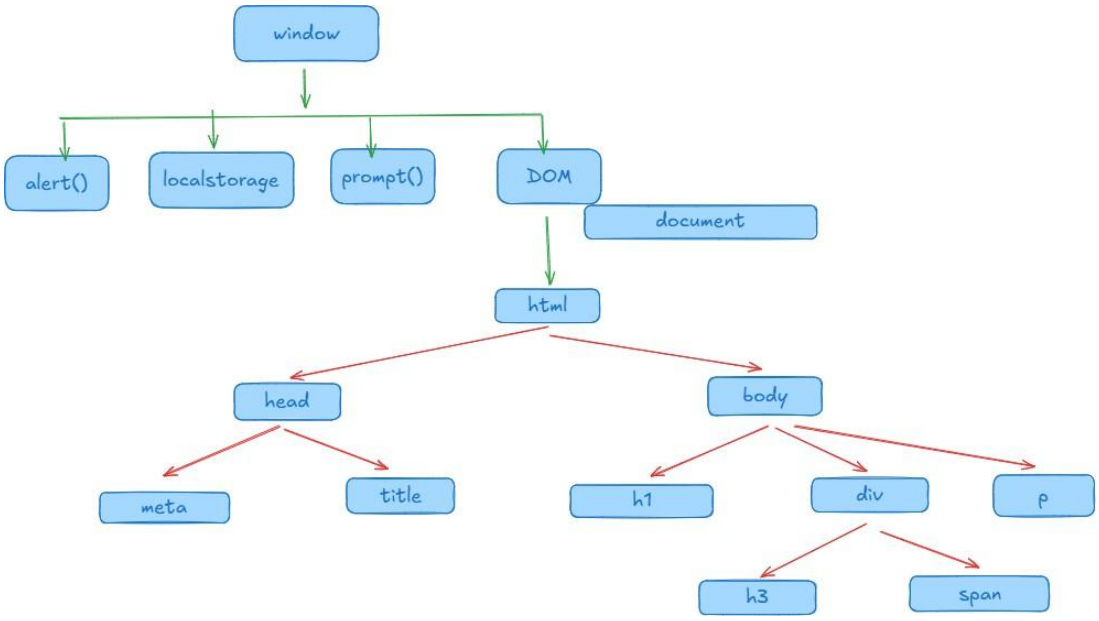
- Async and await are the shortened for promises.
- Async and await both are keywords comes in a pair.
- Async keyword must be used when function is declaration.
- Await keyword is used while fetching data.

BOM [Browser Object Model]

- BOM in JS refers to a set of objects provided by the browser to interact with the browser environment outside of the webpage content.

DOM [Document Object Model]

- DOM is a programming interface for web development. It represents the page so that programs can change the document structure, style and content



DOM Methods

i. **getElementById():**
   - It is used to target the element with the specified id.
   - It returns the element with the specified id if it exists in the document, otherwise it returns null.

ii. **getElementByClassName():**
   - It is used to target all elements with the specified class name.
   - It returns a collection of elements [HTML Collection] with the specified class name if they exist in the document, otherwise it returns an empty collection.

iii. **getElementByTagName():**
   - It is used to target all elements with the specified tag name.

- It returns a collection of elements [HTML Collection] with the specified tag name if they exist in the document, otherwise it returns an empty collection.

iv. **querySelector():**
- It is used to target the first element that matches the specified selector.
- It returns the first element that matches the specified selector if it exists in the document, otherwise it returns null.

v. **getElementByClassName():**
- It is used to target the first element that matches the specified selector.
- It returns a collection of elements [node lists] that match the specified selector if they exist in the document, otherwise it returns an empty list

NOTE:

- **HTML Collection**:
  - It is array like objects but not pure array and object.
  - We cannot perform any of the array or object inbuilt methods for html collection except length.
- **Node Lists**:
  - It is array like objects but not pure array and object.
  - We cannot perform any of the array or object inbuilt methods for node lists except length and forEach().

DOM Properties:

1. **createElement():** used to create an html element.
2. **appendChild():** used to add single html element to the existing dom tree.
3. **append():** used to add multiple html element to existing dom tree.
4. **innerHTML**: used to add multiple html element to existing dom tree.
5. **innerText**: used to add text for the html element to existing dom tree.

Events:

- A JavaScript event is a specific action that occurs within a web page or application, such as clicking on an element, moving the mouse, pressing a key, or loading a page.

Types of Events:

- **Click Events**
  - In JavaScript, a click event is a DOM event that occurs when a user clicks on an button or HTML element.
  - There are several ways to handle click events in JavaScript: Using the onclick HTML attribute or addEventListerner method.
- **Mouse Events**
  - In JavaScript, a mouse event is a DOM event that occurs when a user hover on a HTML element.
  - There are several ways to handle mouse events in JavaScript, Using the mouseover, mouseout etc. HTML attribute or addEventListerner method.
- **Keyboard Events**
  - In JavaScript, a keyboard event is a DOM event that occurs when a user presses any key.
  - There are several ways to handle keyboard events in JavaScript, Using the keydown, keyup, etc. or addEventListerner method.
- **Form Events**
  - Form events in JavaScript are actions that can be detected and responded to when a user interacts with HTML forms and their elements.
  - submit: This event fires when a form is submitted, either by clicking a submit button or pressing Enter within a form field.

localStorage and sessionStorage

- localStorage: localStorage in JavaScript allows web applications to store data locally within the user's browser
  - setItem(): takes a key-value pair and adds it to localStorage.
  - getItem(): takes a key and returns the corresponding value.

- sessionStorage: localStorage in JavaScript allows web applications to store data locally within the user's browser
  - setItem(): takes a key-value pair and adds it to localStorage.
  - getItem(): takes a key and returns the corresponding value.

Project on WEB Tech

"Every line of code you write is a step forward- be proud, enjoy the journey!"

Thank You

Possible Interview Questions on WEB TECH

HTML

1. What Is the difference between HTML elements & HTML tags?
2. What are HTML tags, and how to use them?
3. What are the main features of HTML?
4. What is HTML?
5. What Is HTML formatting?
6. Do all HTML tags have an end tag?
7. Which HTML tags are used to display the data in tabular
8. format?
9. What are HTML attributes & how to use them?
10. What is the basic structure of an HTML template?
11. How many heading tags are there in HTML?
12. How to create an HTML hyperlink?
13. What is the difference between Block Level Element & Inline
14. Element in HTML?
15. What is Semantic HTML?
16. What is the difference between DIV and SPAN In HTML?

CSS

1. What is CSS and why is it used?
2. How do you center a div element in CSS?
3. What is the difference between id and class selectors in CSS?
4. How does Flexbox work in CSS?
5. Explain the CSS box model.
6. What are pseudo-classes and pseudo-elements in CSS?
7. What is the difference between relative, absolute, fixed, and sticky positioning in CSS?

JAVA SCRIPT

1. What is JavaScript?
2. What is JavaScript Promise?
3. What are the characteristics of an array in JavaScript?
4. What is JavaScript array?
5. What is JavaScript function?
6. What is JavaScript DOM?
7. What is JavaScript event?
8. What are the advantages of JavaScript?
9. What is JavaScript used for?
10. What are operators in JavaScript?
11. What is the difference between JavaScript and Java?
12. What was the first name of JavaScript?
13. What is JavaScript ES6?
14. What is a named function in JS?
15. What is an anonymous function?
16. What is a nested function?
17. What is an IIFE function?
18. What is an HOF and Callback function

---

1. What's the difference between a tag and an element?
2. What are attributes in HTML?
3. How do you create hyperlinks? What are absolute vs relative URLs?
4. What's the difference between block-level and inline elements?
5. How do you make a link open in a new tab safely?
6. What is the <label> tag and how does for/id improve accessibility?
7. How HTML is different from CSS?
8. What are semantic HTML elements?
9. How do you add a video in HTML5?
10. How do you add CSS styling in an HTML file?
11. What are the different data types present in javascript?
12. Difference between var, let and const in javascript.
13. Explain logical operators in javascript.
14. Is javascript a statically types or a dynamically typed language?
15. Explain Hoisting in javascript
16. Difference between "==" and "===" in javascript.
17. What is javascript currying in javascript.

18. What is javascript clousure?
19. Difference between java and javascript?
20. Explain Promises with example. Mention Promise static methods and Promise instance methods.
21. Explain output methods in javascript.
22. What is BOM and DOM in javascript?
23. Explain DOM Methods with examples.
24. Explain DOM Properties with examples.
25. Explain DOM Events with examples.
26. Difference between inner Text and inner HTML.
27. Difference between HTML Collection and Node lists
28. Difference between append and appendChild.
29. Explain fetch method in javascript with example.
30. Explain async and await in javascript.
31. What is NaN property in javascript.
32. Explain All Types of functions in javascript with examples.
33. How do you make a multi-line text input?
34. What is the difference between classes and id?

Web Technologies (HTML, CSS & JavaScript)

- **Strong knowledge of HTML5 and CSS3**, including **semantic structure**, **responsive layouts**, and **modern styling techniques** for building visually appealing web pages.
- **Hands-on experience** in **cloning and designing web pages** using HTML and CSS, ensuring **pixel-perfect**, **responsive**, and **user-friendly interfaces**.
- **Good understanding of core JavaScript concepts** such as **hoisting**, **scope**, **closures**, and expertise in **DOM/BOM manipulation**, **asynchronous operations**, **callbacks**, **promises**, and **async/await**.
- **Proficient in client-side data management** using **localStorage**, **sessionStorage**, and **JSON** for efficient data handling and persistence.